

RE Leçon 3 — Exercices : if/else en assembleur

Rappel du pattern

```
cmp  eax, 5      ; compare eax à 5
jle  .else       ; si eax <= 5 → aller à .else
; ... bloc if ...
jmp  .fin
.else:
; ... bloc else ...
.fin:
```

Exercice 1 — Lire un if simple

Compile et désassemble :

```
// ifelse1.c
#include <stdio.h>
int main() {
    int x = 10;
    if (x == 10) {
        printf("oui\n");
    }
    return 0;
}
```

```
gcc -g -O0 -o ifelse1 ifelse1.c
objdump -d -M intel ifelse1 | grep -A 30 "<main>:"
```

Questions :

1. Quelle instruction fait la comparaison ?
 2. Quel saut conditionnel suit ? (`jle` ? `jne` ? `jle` ?)
 3. Pourquoi `jne` et pas `je` — dans quel sens le compilateur génère le saut ?
-

Exercice 2 — if / else

```
// ifelse2.c
#include <stdio.h>
int main() {
    int age = 15;
```

```
if (age >= 18) {
    printf("majeur\n");
} else {
    printf("mineur\n");
}
return 0;
}
```

```
gcc -g -O0 -o ifelse2 ifelse2.c
objdump -d -M intel ifelse2 | grep -A 40 "<main>:"
```

Questions :

1. Trouve le `cmp` — avec quelle valeur compare-t-il `age` ?
2. Quel saut conditionnel est utilisé ? (`jl` ? `jge` ? `jns` ?)
3. Sans changer le code source, modifie `age` dans GDB pour forcer le chemin "majeur" :

```
gdb ./ifelse2
(gdb) break main
(gdb) run
(gdb) set variable age = 18
(gdb) continue
```

Exercice 3 — if / else if / else

```
// ifelse3.c
#include <stdio.h>
int main() {
    int note = 12;
    if (note >= 16) {
        printf("Très bien\n");
    } else if (note >= 10) {
        printf("Passable\n");
    } else {
        printf("Insuffisant\n");
    }
    return 0;
}
```

```
gcc -g -O0 -o ifelse3 ifelse3.c
objdump -d -M intel ifelse3 | grep -A 60 "<main>:"
```

Questions :

1. Combien de `cmp` tu vois dans l'assembleur ?
2. Combien de sauts conditionnels ?
3. Dessine le schéma de flux :

```

    cmp note, 16
    |
    jl -- Très bien
    |
    cmp note, 10
    |
    ...

```

Exercice 4 — Condition avec AND (&&)

```

// ifelse4.c
#include <stdio.h>
int main() {
    int x = 5;
    int y = 8;
    if (x > 3 && y < 10) {
        printf("les deux conditions vraies\n");
    } else {
        printf("au moins une fausse\n");
    }
    return 0;
}

```

```

gcc -g -O0 -o ifelse4 ifelse4.c
objdump -d -M intel ifelse4 | grep -A 50 "<main>:"

```

Questions :

1. Combien de `cmp` pour évaluer `x > 3 && y < 10` ?
2. Comment le compilateur court-circuite la 2ème condition si la 1ère est fausse ?

Exercice 5 — Condition avec OR (||)

```

// ifelse5.c
#include <stdio.h>
int main() {
    int a = 2;

```

```
int b = 9;
if (a == 0 || b == 9) {
    printf("au moins une vraie\n");
} else {
    printf("toutes fausses\n");
}
return 0;
}
```

```
gcc -g -O0 -o ifelse5 ifelse5.c
objdump -d -M intel ifelse5 | grep -A 50 "<main>:"
```

Questions :

1. Comment `||` est-il traduit différemment de `&&` en asm ?
2. Quel saut est utilisé pour le court-circuit du `||` ?

Exercice 6 — Crackme : bypasser un if avec GDB

```
// bypass.c
#include <stdio.h>
#include <string.h>

int est_premium(int code) {
    if (code == 9999) {
        return 1;
    }
    return 0;
}

int main() {
    int code;
    printf("Code premium: ");
    scanf("%d", &code);

    if (est_premium(code)) {
        printf("Accès premium accordé\n");
    } else {
        printf("Accès refusé\n");
    }
    return 0;
}
```

```
gcc -g -O0 -o bypass bypass.c
```

Objectif : obtenir "Accès premium accordé" **sans connaître le code**.

Méthode 1 — Modifier la variable :

```
gdb ./bypass
(gdb) break est_premium
(gdb) run
# entre n'importe quel code
(gdb) set variable code = 9999
(gdb) continue
```

Méthode 2 — Modifier le registre de retour :

```
gdb ./bypass
(gdb) break est_premium
(gdb) run
# entre n'importe quel code
(gdb) finish          # exécuter jusqu'au return
(gdb) set $rax = 1    # forcer le retour à 1 (= true)
(gdb) continue
```

Méthode 3 — Lire l'asm et trouver le code :

```
objdump -d -M intel bypass | grep -A 20 "<est_premium>:"
# cherche le cmp avec la valeur magique
```

La méthode 2 est la plus puissante — elle fonctionne même sans connaître la logique interne de la fonction.

Exercice 7 — Identifier le if dans un binaire stripped

Compile **sans symboles** pour simuler un vrai binaire :

```
// stripped.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[20];
    printf("Entrez un mot: ");
    scanf("%19s", input);

    if (strlen(input) > 5) {
        printf("Trop long\n");
    } else {
```

```
        printf("OK\n");
    }
    return 0;
}
```

```
gcc -O0 -s -o stripped stripped.c # -s = strip les symboles
objdump -d -M intel stripped | grep -A 50 "<main>:"
```

Questions :

1. Sans les noms de variables, identifie quand même le `cmp`
2. Avec quelle valeur compare-t-il ? (c'est le seuil 5)
3. Quel saut est utilisé ?

Checkpoint

- ☐ Ex 1 : trouvé le `cmp` + compris pourquoi `jne` et pas `je`
- ☐ Ex 2 : modifié `age` dans GDB pour bypasser le if
- ☐ Ex 3 : compté les `cmp` dans un if/else if/else
- ☐ Ex 4 : compris comment `&&` génère deux `cmp` + court-circuit
- ☐ Ex 5 : compris la différence `&&` vs `||` en asm
- ☐ Ex 6 : bypassé un if via 3 méthodes GDB différentes
- ☐ Ex 7 : identifié le `cmp` dans un binaire sans symboles